



Module Component Report

Name : LONG Yuhan Subject Code : EIE2903
Student ID : 24105998d Module Code : TM1117
Programme Code : 46409-AIE Module Component : IoT and AI
Programme Title : Artificial Intelligence & Information Engineering
Dates of Attendance: June 26th -July 4th, 2025

Important note: This report plays an important part in your IC subject. It is a permanent record of what **you** consider important and have learned, which includes but not limited to events occurred, observations made, and problems identified (and solved). **Plagiarism will result in module fail.**

Introduction

(Give the general overview of the module component e.g. expected learning outcomes, run-down, etc.)

Throughout this subject, I have gained a comprehensive exploration of advanced system applications and practice for the Internet and multimedia with many hands-on practices. Key components like GitHub, MQTT, and JSON protocols, database management for sensor data, implementation of front-end application design with Django, and other AI applications are all included in the foundational training of this course. With both theoretical learning and practical tasks, I have gained essential capabilities to design IoT solutions connecting Raspberry Pi sensors to web interfaces, as well as AI applications for data analysis and image recognition. These elements together helped me create a comprehensive grasp of modern IoT systems, spanning from the device layer to the user interface. The report will delve into my learning experiences in technical aspects, safety measures, and an overview of the entire training of TM1117.

Technical Content

(Describe technically ONE to TWO most important content matters you have learned in this module component, e.g. the procedures, tools & equipment used, factors affecting the quality, etc.)

IOT

MQTT, JSON, and SQL

MQTT (Message Queuing Telemetry Transport) is a popular messaging protocol. It helps machines and IoT devices communicate. MQTT works on TCP/IP sockets, WebSockets, and SSL. It uses a “topic” system. Publishers send messages to a server on a specific topic. Subscribers can choose to follow one or more topics. The MQTT

broker (the server) sends messages to the subscribers that follow the right topics. For example, in one of our projects, temperature sensors were publishers and laptops were subscribers. When sensors sent temperature data to the MQTT Broker, the server sent that data to all the laptops.

JSON (JavaScript Object Notation) is a simple data interchange format. It's easy for people to read and write. In IoT, JSON is useful because it can describe itself and works well with programming languages.

SQL (Structured Query Language) is a standard language for relational database management. It lets users create, query, update, and delete data in tables, with key features like using SELECT to retrieve data, INSERT to add records, and UPDATE and DELETE to modify and remove data. Tables in SQL store data in rows and columns, and relationships between tables can be established through keys. A primary key is a column or set of columns in a table that uniquely identifies each row, must contain unique non-NULL values, and is crucial for joining tables and ensuring data integrity—for example, `customer_id` serves as the primary key in the CUSTOMERS table. A fundamental feature of SQL is JOIN, which enables querying data from multiple tables by matching related values. This is essential because real-world data is often distributed across multiple tables. In our following projects, Django's ORM dynamically converts Python code into SQL queries. For example, actions such as filtering, sorting, and aggregating model objects are translated into efficient SQL statements for database management.

Django Fundamentals

We began by creating a virtual environment using Anaconda and initialized a project with `django-admin startproject`. Once the environment was ready, we initialized the entire project framework with the `django-admin startproject` command.

Key configurations included setting the Hong Kong timezone (`TIME_ZONE = 'Asia/Hong_Kong'`) in `settings.py` and registering apps like `blog`, `IoT`, and `dashboard` in the `INSTALLED_APPS` list.

Next, defining data models in `models.py` was a crucial step. Models such as `Event`, which was for IoT data, and `Entry`, which was for blog content, were designed in `models.py`, featuring fields like `CharField`, `DecimalField`, and `DateTimeField`. After designing the models, database synchronization was achieved via `python manage.py makemigrations` and `migrate`, with SQLite as the default DB.

To facilitate data management, the admin interface was enabled. This was done by registering models in `admin.py` and creating a superuser. URL routing (`urls.py`) mapped endpoints to views, while templates (HTML files in `templates/` directories) rendered dynamic content using Django template tags such as `{% for %}` and `{{ variables }}`. Finally, for the look and formatting of the website, Bootstrap CSS was integrated.

Django IoT App

Technical Content (...Continued)

When the previous steps were finished, we developed a Python MQTT client named `iot_mqtt.py`. The client connected to the broker `ia.ic.polyu.edu.hk` on port 1883 and subscribed to the `iot/sensor` topic. The data arrived in JSON format, like `{"id": "11", "loc": "w502g", "temp": "25.0"}`. Incoming JSON payloads were decoded and saved to the Event model via `p.save()`. In the `views.py` file, we implemented a query to retrieve the sensor data. By using `Event.objects.order_by("-date_created")`, we could fetch the most recent data first. After applying Bootstrap CSS in template, the data was ready for display. Also, the template was modified to set temperature thresholds triggered color-coded alerts. If the temperature was above 29°C, the text would turn red to indicate a potentially critical situation. On the other hand, when the temperature dropped below 21°C, the text would change to blue.

Django Dashboard

To provide the visualization tool with data, we started by serializing the sensor data from the Event model in `views.py`. I utilized `django.core.serializers` to convert the data into JSON format, making it easily consumable by the frontend. This serialized data was then made available via the `/dashboard/temp_data` endpoint. The frontend employed AJAX (`$.ajax()`) to fetch data every 5 seconds, which can be configurable via `setTimeout()`.

Several key components contributed to the effective operation of the dashboard. First, a `processData()` function parsed JSON and extracted fields for Flexmonster compatibility. When it came to visualization, we configured Flexmonster to display visual elements in conditional formatting. Moreover, to enable real-time updates, I utilized `flexmonster.updateData()`.

Artificial Intelligence

Supervised learning uses labeled data for prediction, like classifying images or predicting house prices with algorithms such as linear regression and decision trees. Unsupervised learning finds patterns in unlabeled data, with K-means for clustering and PCA for dimensionality reduction.

Deep learning, a branch of machine learning, uses deep neural networks to automatically learn data representations. It eliminates the need for manual feature engineering, extracting complex features from raw data. CNNs are applied to image recognition, while RNNs and LSTMs handle sequential data. Deep learning has

transformed multiple fields, achieving great success in tasks like speech and language processing.

CNN

CNN, short for convolutional neural network, was implemented for image classification and object detection using hierarchical feature extraction. They use shared weights and sparse connections to get spatial features. The main parts are convolutional layers to extract features, pooling layers to shrink dimensions, activation functions to add non-linearity, and fully connected layers for classification. They are used for tasks like object classification, detection like in the R-CNN series, and segmentation. Advanced models like YOLO and SSD can detect objects in real-time by processing images all at once.

In Task 2 (MNIST Classification), we implemented a CNN with Conv2D(32, (3,3)) and MaxPooling2D((2,2)) layers for feature extraction, followed by dense layers with categorical_crossentropy loss. In Task 3 (Object Detection), we compared different architectures: the MobileNetV2 SSD had 75.9K parameters and achieved 22ms inference time, while the InceptionResNet Faster R-CNN had 105.8K parameters and was more accurate but slower. We used LabelImg for bounding box annotation and measured precision/recall on furniture datasets. For Task 4 (Helmet Detection), we added domain-specific logic. The model detected "Helmet" and "Human Hair" or "Human Face" co-occurrences, implemented bounding box intersection checks, and output "With Helmet" labels only when both classes were spatially proximate. This safety application achieved an approximately 50% accuracy in construction site imagery.

RNN

Recurrent Neural Networks (RNNs) are designed for sequence data, featuring loops that maintain historical context through a hidden state. They handle sequences step-by-step but face vanishing/exploding gradient issues with long data. Solutions like LSTMs and GRUs use gating mechanisms (forget/input/output gates) to retain long-term dependencies. Applications include speech recognition, video activity detection, stock prediction, and machine translation.

In Task 5, we focus on time-series forecasting using various neural networks. For single-step models, we implement a linear network with a simple Dense layer, a dense network with two 64-unit ReLU hidden layers, a CNN using Conv1D with 32 filters, and an LSTM with 32 units. The LSTM shows the best performance with a loss of 0.0535, outperforming others due to its ability to handle long-term dependencies. In multi-

step forecasting, we build linear, dense, CNN, and LSTM models. The dense model with 512-unit hidden layers achieves a loss of 0.2870, slightly better than the linear model (0.2982), while the LSTM (0.3407) and CNN (0.3308) perform relatively worse. Different networks show varying performance because linear/dense networks lack sequential context, CNNs excel at local features but struggle with long-range dependencies, and LSTMs use gating mechanisms to retain historical information. Thus, LSTMs are the most suitable for time-series forecasting as they effectively model complex temporal patterns and mitigate gradient issues.

Safety

(Give the outline of the safety measures taken)

Safety has always been the priority in the training. Here are 3 safety measures:

1. Do not eat or drink during the training. Wear long trousers and sneakers instead of shorts or slippers. Also, take a break after a long period of work to relieve the strain.
2. Electrical safety is vital for AI hardware. Check cables for fraying, use grounded outlets, and avoid overloading circuits to prevent hazards. Turn off and unplug equipment when not in use or during maintenance.
3. Data and information security is also important. Always remember to back up the files before leaving the class. Regularly back up data to prevent loss.

Observation/Conclusion

(Describe the knowledge, skills, and experiences you have acquired and how they are related to your discipline. Suggest ways to further improve the module component)

Throughout this module, I have gained comprehensive skills in advanced IoT and multimedia systems, combining technical expertise with practical application. Key learnings include MQTT/JSON for device communication, SQL for data management, and Django for web development, enabling end-to-end solutions linking Raspberry Pi sensors to dynamic interfaces. In AI, I applied CNNs for image classification (MNIST, object detection with YOLO/SSD) and RNN/LSTM for time-series forecasting, leveraging their strengths in feature extraction and sequential analysis. These competencies directly support my discipline, bridging hardware with software for real-world IoT scenarios like smart environments and construction monitoring.

Some topics can be challenging during my study, but I could overcome the difficulties by discussing with my partner and others, asking the teacher for help with some IT problems, and consulting GenAI to analyze and understand some code. Overall, not

only did I improve my professional capabilities, but I also learned to better cooperate with others to solve a common problem.

(For Marker Only)

Comment:	Assessed by:
	(BLOCK LETTER) Signature:
Mark: (out of 100)	Date: